VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

OPERATINGSYSTEMS

Submitted by

RUSHIL SATHISH(1WA24CS243)

in partial fulfillment for the award of the degree of
BACHELOROFENGINEERING
in
COMPUTER SCIENCEAND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Feb-2026 to June-2026

B. M. S. College of Engineering,
BullTempleRoad,Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by RUSHIL SATHISH(1WA24CS243), who is a Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026.
The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Faculty Incharge Name	Dr. Kavitha Sooda
Assistant Professor	Professor and Head
Department of CSE	Department of CSE
BMSCE, Bengaluru	BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF c) Priority d) Round Robin	1
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	25
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate-Monotonic b) Earliest-deadline First c) Proportional scheduling	30
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	41
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	47
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	51
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	56

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating Systems.
C04	Conduct practical experiments to implement the functionalities of Operating systems.

Program 1

Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.
a)FCFS
b)SJF
c)Priority
d)Round Robin

a) FCFS

Code:

```
#include <stdio.h>

intmain()
{
   printf("Process Scheduling - FCFS\n");
   printf("RUSHIL
   SATHISH1WA24CS243\n");
   intn;
   printf("\nEnter number of processes: ");
   scanf("%d", &n);

   //arrival time = aTime        burst time = bTime
   //completion time = cTime
   //turnaround time = taTime      wait time = wTime
   //start time = sTime          response time = rTime

   intpID[n], aTime[n], bTime[n],
      cTime[n], taTime[n], wTime[n], sTime[n],
      rTime[n];
   intcurrTime = 0, totalTAT = 0, totalWT = 0, temp;

   printf("\nEnter AT BT: \n");
   for(int i = 0; i < n; i++)
   {
      pID[i] = i + 1;
```

```
    printf("\tP%d: ", pID[i]);
    scanf("%d%d",&aTime[i],&bTime[i]);
}

// sorting by AT
for (int i = 0; i < n; i++)
{
    for(intj=0;j<n-i-1;j++)
    {
        if(aTime[j]>aTime[j+1])     // if process enters after current process
        {
            // swap PID
            temp = pID[j];
            pID[j] = pID[j + 1];
            pID[j + 1] = temp;

            // swap AT
            temp = aTime[j];
            aTime[j]=aTime[j+1];
            aTime[j + 1] = temp;

            // swap BT
            temp = bTime[j];
            bTime[j]=bTime[j+1];
            bTime[j + 1] = temp;
        }
    }
}

for (int i = 0; i < n; i++)
{
    if (currTime < aTime[i])
        currTime = aTime[i];

    sTime[i] = currTime;

    // completion time
    cTime[i]=currTime+bTime[i];

    // turnaround time
    taTime[i]=cTime[i]-aTime[i];
```

```
        totalTAT += taTime[i];

        // wait time
        wTime[i]=taTime[i]-bTime[i];
        totalWT += wTime[i];

        // response time
        rTime[i]=sTime[i]-aTime[i];

        currTime = cTime[i];
    }

    printf("\nPID\tAT\tBT\tCT\tTAT\t WT\t RT\n");
    for (int j = 1; j < n + 1; j++)
    {
        for (int i = 0; i < n; i++)
        {
            if (pID[i] == j)
            {
                printf("P%d\t%d\t%d\t %d\t %d\t %d\t %d\n",
                       pID[i],aTime[i],bTime[i], cTime[i],
                       taTime[i],wTime[i], rTime[i]);
            }
        }
    }

    printf("Avg.TAT=%.2f\n",((float)totalTAT / n));
    printf("Avg.WT=%.2f\n",((float)totalWT / n));

    return 0;
}
```

Output:

```
Process Scheduling - FCFS


Enter number of processes: 5

Enter AT BT:
        P1: 0 2
        P2: 1 2
        P3: 5 3
        P4: 6 4
        P5: 8 1
        P4: 6 4
        P5: 8 1

PID     AT      BT      CT      TAT     WT      RT
P1      0       2       2       2       0       0
P2      1       2       4       3       1       1
P3      5       3       8       3       0       0
P4      6       4       12      6       2       2
P5      8       1       13      5       4       4
Avg. TAT = 3.80
Avg. WT = 1.40
```

b) SJF

Code:

```
#include <stdio.h>

int main()
{
   printf("ProcessScheduling - SJF\n");
   printf("RUSHIL
   SATHISH1WA24CS243\n");
   int n;
   printf("\nEnternumber of processes: ");
   scanf("%d", &n);

   //arrivaltime=aTime          burst time = bTime
   //completiontime= cTime
   //turnaroundtime= taTime       wait time = wTime
   //starttime=sTime             response time = rTime

   intpID[n],aTime[n], bTime[n],
      cTime[n],taTime[n], wTime[n], sTime[n],
      rTime[n],done[n];
   intcurrTime=0,totalTAT = 0, totalWT = 0, temp,
      completed = 0;

   printf("\nEnterATBT: \n");
   for(inti=0;i<n;i++)
   {
      pID[i] = i + 1;

      printf("\tP%d:",pID[i]);
      scanf("%d%d",&aTime[i], &bTime[i]);

      done[i] = 0;
   }

   // sorting by AT
   for(inti=0;i<n;i++)
   {
      for(intj=0;j<n - i - 1; j++)
      {
```

```
        if(aTime[j] > aTime[j + 1])  // if process enters after current process
        {
            //swapPID
            temp = pID[j];
            pID[j] = pID[j+1];
            pID[j + 1]=temp;

            // swap AT
            temp =aTime[j];
            aTime[j]=aTime[j+1];
            aTime[j + 1] = temp;

            // swap BT
            temp =bTime[j];
            bTime[j]=bTime[j+1];
            bTime[j + 1] = temp;
        }
    }
}

while(completed != n)
{
    intminTime = 99999;
    intcurrPrc = -1;

    for(inti= 0; i < n; i++)
    {
        if(aTime[i] <= currTime && !done[i])  // if uncompleted process arrives before current time
        {
            if(bTime[i] < minTime)  //ifprocesshas shortest exec time
            {
                minTime = bTime[i]; //assignlowest priority
                currPrc = i;        //assigncurrentprocess
            }
        }
    }

    //accountingforCPU idle time
    if(currPrc == -1)
    {
        currTime++;
        continue;
```

```c
        }

        sTime[currPrc]=currTime;

        // completion time
        cTime[currPrc]=sTime[currPrc] + bTime[currPrc];

        // turnaround time
        taTime[currPrc]=cTime[currPrc] - aTime[currPrc];
        totalTAT+=taTime[currPrc];

        // wait time
        wTime[currPrc]=taTime[currPrc] - bTime[currPrc];
        totalWT+=wTime[currPrc];

        // response time
        rTime[currPrc]=sTime[currPrc] - aTime[currPrc];

        currTime=cTime[currPrc];

        done[currPrc] = 1;
        completed++;
    }

    printf("\nPID\tAT\tBT\t CT\tTAT\t WT\t RT\n");
    for(intj=1;j<n+1;j++)
    {
        for(inti=0;i<n;i++)
        {
            if (pID[i] == j)
            {
                printf("P%d\t%d\t %d\t %d\t %d\t %d\t %d\n",
                    pID[i],aTime[i], bTime[i], cTime[i],
                    taTime[i],wTime[i], rTime[i]);
            }
        }
    }

    printf("Avg.TAT=%.2f\n", ((float)totalTAT / n));
    printf("Avg.WT=%.2f\n", ((float)totalWT / n));

    return 0;
```

}

Output:

```
Process Scheduling - SJF


Enter number of processes: 5

Enter AT BT:
        P1: 0 6
        P2: 0 8
        P3: 0 7
        P4: 0 3
        P5: 0 2

PID      AT      BT      CT      TAT      WT      RT
P1       0       6       11       11      5       5
P2       0       8       26       26      18      18
P3       0       7       18       18      11      11
P4       0       3       5        5       2       2
P5       0       2       2        2       0       0
Avg. TAT = 12.40
Avg. WT = 7.20
```

c) SRTF

Code:

```
#include <stdio.h>

int main()
{
   printf("ProcessScheduling - SRTF\n");
   printf("RUSHIL
   SATHISH1WA24CS243\n");
   int n;
   printf("\nEnternumber of processes: ");
   scanf("%d", &n);

   //arrivaltime=aTime           burst time = bTime
   //completiontime= cTime
   //turnaroundtime= taTime        wait time = wTime
   //starttime=sTime              response time = rTime

   intpID[n],aTime[n], bTime[n],
      cTime[n],taTime[n], wTime[n], sTime[n],
      rTime[n],remaining[n];
   intcurrTime=0,totalTAT = 0, totalWT = 0, temp,
      completed = 0;

   printf("\nEnterATBT: \n");
   for(inti=0;i<n;i++)
   {
      pID[i] = i + 1;

      printf("\tP%d:",pID[i]);
      scanf("%d%d",&aTime[i], &bTime[i]);
   }

   // sorting by AT
   for(inti=0;i<n;i++)
   {
      for(intj=0;j<n - i - 1; j++)
      {
         if(aTime[j]>aTime[j + 1])    // if process enters after current process
         {
```

```
            // swap PID
            temp = pID[j];
            pID[j] = pID[j+1];
            pID[j + 1]=temp;

            // swap AT
            temp = aTime[j];
            aTime[j]=aTime[j+1];
            aTime[j + 1] = temp;

            // swap BT
            temp = bTime[j];
            bTime[j]=bTime[j+1];
            bTime[j + 1] = temp;
         }
      }
   }

   for(inti=0; i < n; i++)
      remaining[i] = bTime[i];

   while(completed != n)
   {
      intminTime = 99999; // init minimum time
      intcurrPrc = -1;    // init current process

      for(inti= 0; i < n; i++)
      {
         if(aTime[i] <= currTime && remaining[i] > 0)  // if uncompleted process arrives before
current time
         {
            if(remaining[i] < minTime)  // if process has remaining time
            {
               minTime = remaining[i]; // assign lowest priority
               currPrc = i;           // assign current process
            }
         }
      }

      //accounting for CPU idle time
      if (currPrc == -1)
      {
```

```
        currTime++;
        continue;
    }

    // if process is entering for first time
    if(remaining[currPrc]==bTime[currPrc])
        sTime[currPrc]=currTime;

    remaining[currPrc]--;

    // if no remaining time left
    if(remaining[currPrc]==0)
    {
        // completion time
        cTime[currPrc]=currTime + 1;

        // turnaround time
        taTime[currPrc]=cTime[currPrc] - aTime[currPrc];
        totalTAT+=taTime[currPrc];

        // wait time
        wTime[currPrc]=taTime[currPrc] - bTime[currPrc];
        totalWT+=wTime[currPrc];

        // response time
        rTime[currPrc]=sTime[currPrc] - aTime[currPrc];

        completed++;
    }

    currTime++;
}

printf("\nPID\tAT\tBT\tCT\tTAT\t WT\t RT\n");
for (int j = 1; j < n + 1; j++)
{
    for (int i = 0; i < n; i++)
    {
        if (pID[i] == j)
        {
            printf("P%d\t%d\t%d\t %d\t %d\t %d\t %d\n",
                   pID[i],aTime[i],bTime[i], cTime[i],
```

```
                taTime[i], wTime[i], rTime[i]);
            }
        }
    }

    printf("Avg. TAT = %.2f\n", ((float)totalTAT / n));
    printf("Avg. WT = %.2f\n", ((float)totalWT / n));

    return 0;
}
```

Output:

```
Process Scheduling - SRTF


Enter number of processes: 5

Enter AT BT:
        P1: 2 1
        P2: 1 5
        P3: 4 1
        P4: 0 6
        P5: 2 3

PID     AT      BT      CT      TAT     WT      RT
P1      2       1       3       1       0       0
P2      1       5       16      15      10      10
P3      4       1       5       1       0       0
P4      0       6       11      11      5       0
P5      2       3       7       5       2       1
Avg. TAT = 6.60
Avg. WT = 3.40
```

d) Priority Scheduling (Non-pre-emptive)

Code:

```
#include <stdio.h>

intmain()
{
  printf("Priority  Scheduling  (Non-Preemptive)\n");
  printf("Lower the number, higher the priority.\n");
  printf("RUSHIL SATHISH1WA24CS243\n");

  intn;
  printf("\nEnter number of processes: ");
  scanf("%d", &n);

  //arrival time = aTime        burst time = bTime
  //priority = prio             completion time = cTime
  //turnaround time = taTime     wait time = wTime
  //start time = sTime          response time = rTime

  intpID[n], aTime[n], bTime[n], prio[n],
    cTime[n], taTime[n], wTime[n], sTime[n],
    rTime[n], done[n];
  intcurrTime = 0, totalTAT = 0, totalWT = 0, completed = 0;

  printf("\nEnter AT BT P: \n");
  for(int i = 0; i < n; i++)
  {
    pID[i] = i + 1;

    printf("\tP%d: ", pID[i]);
    scanf("%d %d %d", &aTime[i], &bTime[i], &prio[i]);

    done[i] = 0;
  }

  while (completed < n)
  {
    int minPrio = 999; // init minimum priority
    int currPrc = -1; // init current process
```

```
    for (int i = 0; i < n; i++)
    {
        if(aTime[i]<=currTime&& !done[i])     // if uncompleted process arrives before current time
        {
            if(prio[i]<minPrio)    //ifprocessisof lower priority
            {
                minPrio=prio[i];   //assignlowestpriority
                currPrc=i;         //assigncurrentprocess
            }
        }
    }

    //accountingforCPUidletime
    if (currPrc == -1)
    {
        currTime++;
        continue;
    }

    sTime[currPrc] = currTime;

    // completion time
    cTime[currPrc]=sTime[currPrc] + bTime[currPrc];

    // turnaround time
    taTime[currPrc]=cTime[currPrc] - aTime[currPrc];
    totalTAT+=taTime[currPrc];

    // wait time
    wTime[currPrc]=taTime[currPrc] - bTime[currPrc];
    totalWT += wTime[currPrc];

    // response time
    rTime[currPrc]=sTime[currPrc] - aTime[currPrc];

    currTime = cTime[currPrc];

    done[currPrc] = 1;
    completed++;
}

printf("\nPID\tAT\tBT\tP\tCT\t TAT\t WT\t RT\n");
```

```c
    for (int j = 1; j < n + 1; j++)
    {
        for (int i = 0; i < n; i++)
        {
            if (pID[i] == j)
            {
                printf("P%d\t %d\t %d\t %d\t %d\t %d\t %d\t %d\n",
                pID[i], aTime[i], bTime[i], prio[i], cTime[i],
                taTime[i], wTime[i], rTime[i]);
            }
        }
    }

    printf("Avg. TAT = %.2f\n", ((float)totalTAT / n));
    printf("Avg. WT = %.2f\n", ((float)totalWT / n));

    return 0;
}
```

Output:

```
Priority Scheduling (Non-Preemptive)
Lower the number, higher the priority.


Enter number of processes: 5

Enter AT BT P:
        P1: 0 3 5
        P2: 2 2 3
        P3: 3 5 2
        P4: 4 4 4
        P5: 6 1 1

PID     AT      BT      P       CT      TAT     WT
P1      0       3       5       3       3       0
P2      2       2       3       11      9       7
P3      3       5       2       8       5       0
P4      4       4       4       15      11      7
P5      6       1       1       9       3       2
Avg. TAT = 6.20
Avg. WT = 3.20
```

e) Priority Scheduling (Pre-emptive)

Code:

```
#include <stdio.h>

intmain()
{
   printf("Priority Scheduling (Preemptive)\n");
   printf("Lower the number, higher the priority.\n");
   printf("RUSHIL SATHISH1WA24CS243\n");

   intn;
   printf("\nEnter number of processes: ");
   scanf("%d", &n);

   //arrival time = aTime        burst time = bTime
   //priority = prio             completion time = cTime
   //turnaround time = taTime      wait time = wTime
   //start time = sTime           response time = rTime

   intpID[n], aTime[n], bTime[n], prio[n],
      cTime[n], taTime[n], wTime[n], sTime[n],
      rTime[n], remaining[n];
   intcurrTime = 0, totalTAT = 0, totalWT = 0, completed = 0;

   printf("\nEnter AT BT P: \n");
   for(int i = 0; i < n; i++)
   {
      pID[i] = i + 1;

      printf("\tP%d: ", pID[i]);
      scanf("%d %d %d", &aTime[i], &bTime[i], &prio[i]);

      remaining[i] = bTime[i];
   }

   while (completed < n)
   {
      int minPrio = 99999; // init minimum priority
      int currPrc = -1;    // init current process
```

```
        for(inti= 0; i < n; i++)
        {
          if(aTime[i] <= currTime && remaining[i] > 0)  // if uncompleted process arrives before
current time
            {
              if(prio[i] < minPrio)  //ifprocesshaslowerpriority
              {
                minPrio = prio[i]; // assign lowest priority
                currPrc = i;       // assign current process
              }
            }
        }

        //accountingforCPU idle time
        if(currPrc == -1)
        {
          currTime++;
          continue;
        }

        // if process is entering for first time
        if(remaining[currPrc] == bTime[currPrc])
          sTime[currPrc] = currTime;

        remaining[currPrc]--;

        // if no remaining time left
        if(remaining[currPrc]==0)
        {
          //completion time
          cTime[currPrc] =currTime + 1;

          // turnaround time
          taTime[currPrc] =cTime[currPrc] - aTime[currPrc];
          totalTAT +=taTime[currPrc];

          // wait time
          wTime[currPrc] = taTime[currPrc] - bTime[currPrc];
          totalWT +=wTime[currPrc];

          // response time
          rTime[currPrc]=sTime[currPrc] - aTime[currPrc];
```

```
            completed++;
        }

        currTime++;
    }

    printf("\nPID\tAT\tBT\t P \t CT\t TAT\t WT\t RT\n");
    for(intj=1;j<n+1;j++)
    {
        for(inti=0;i<n;i++)
        {
            if (pID[i] == j)
            {
                printf("P%d\t%d\t %d\t %d\t %d\t %d\t %d\t %d\n",
                pID[i],aTime[i], bTime[i], prio[i], cTime[i],
                taTime[i],wTime[i], rTime[i]);
            }
        }
    }

    printf("Avg.TAT=%.2f\n", ((float)totalTAT / n));
    printf("Avg.WT=%.2f\n", ((float)totalWT / n));

    return 0;
}
```

Output:

```
Priority Scheduling (Preemptive)
Lower the number, higher the priority.


Enter number of processes: 5

Enter AT BT P:
        P1: 0 8 3
        P2: 1 2 4
        P3: 3 4 4
        P4: 4 1 5
        P5: 5 6 2

PID     AT      BT      P       CT      TAT     WT
P1      0       8       3       14      14      6
P2      1       2       4       16      15      13
P3      3       4       4       20      17      13
P4      4       1       5       21      17      16
P5      5       6       2       11      6       0
Avg. TAT = 13.80
Avg. WT = 9.60
```

f) Round Robin Scheduling

Code:

```
#include <stdio.h>

intmain()
{
    printf("Round Robin Scheduling\n");
    printf("RUSHIL
    SATHISH1WA24CS243\n");
    intn;
    printf("\nEnter number of processes: ");
    scanf("%d", &n);

    //arrival time = aTime        burst time = bTime
    //completion time = cTime      turnaround time = taTime
    //wait time = wTime           start time = sTime
    //response time = rTime        ready queue = readyQ

    intpID[n], aTime[n], bTime[n], cTime[n],
        taTime[n], wTime[n], sTime[n],
        rTime[n], remaining[n], inQueue[n];
    intcurrTime = 0, totalTAT = 0, totalWT = 0,
        completed = 0, timeQuant = 0;
    intreadyQ[n], front = 0, rear = -1, qSize = 0;

    printf("\nEnter AT BT: \n");
    for(int i = 0; i < n; i++)
    {
        pID[i] = i + 1;

        printf("\tP%d: ", pID[i]);
        scanf("%d %d", &aTime[i], &bTime[i]);

        remaining[i] = bTime[i];
    }

    printf("Enter time quantum: ");
    scanf("%d", &timeQuant);

    for(int i = 0; i < n; i++)
```

```
      inQueue[i] = 0;

   while(completed != n)
   {
      for(inti= 0; i < n; i++)
      {
         if(aTime[i] <= currTime && remaining[i] > 0)  // if uncompleted process arrives before
current time
         {
            if(!inQueue[i]) // if process not in queue
            {
               rear = (rear + 1) % n;
               readyQ[rear] = i; // enqueue process
               inQueue[i] = 1;   //flagprocessaspartofqueue
               qSize++;
            }
         }
      }

      //accountingfor CPU idle time
      if(qSize== 0)   // if ready queue empty
      {
         currTime++;
         continue;
      }

      // dequeue process
      intcurrPrc = readyQ[front];
      front=(front + 1)%n;
      inQueue[currPrc] = 0;
      qSize--;

      // if process is entering for first time
      if(remaining[currPrc] == bTime[currPrc])
         sTime[currPrc] = currTime;

      if(remaining[currPrc] < timeQuant)  //ifless remaining time than quantum
      {
         currTime += remaining[currPrc]; //complete entire process
         remaining[currPrc] = 0;
      }
      else
```

```
{
   currTime += timeQuant; //completequantumamount of process
   remaining[currPrc] -= timeQuant;
}

for(intj= 0; j < n; j++)
{
   if(aTime[j] <= currTime && remaining[j] > 0)
   {
      if(!inQueue[j] && j != currPrc)
      {
         rear = (rear + 1) % n;
         readyQ[rear] = j;
         inQueue[j] = 1;
         qSize++;
      }
   }
}

// if no remaining time left
if(remaining[currPrc]==0)
{
   //completion time
   cTime[currPrc] =currTime;

   // turnaround time
   taTime[currPrc] =cTime[currPrc] - aTime[currPrc];
   totalTAT +=taTime[currPrc];

   // wait time
   wTime[currPrc] = taTime[currPrc] - bTime[currPrc];
   totalWT +=wTime[currPrc];

   // response time
   rTime[currPrc]=sTime[currPrc] - aTime[currPrc];

   completed++;
}
else
{
   rear=(rear + 1) % n;
   readyQ[rear] = currPrc;
```

```
            inQueue[currPrc] = 1;
            qSize++;
        }
    }

    printf("\nPID\t AT\t BT\t CT\t TAT\t WT\t RT\n");
    for (int j = 1; j < n + 1; j++)
    {
        for (int i = 0; i < n; i++)
        {
            if (pID[i] == j)
            {
                printf("P%d\t %d\t %d\t %d\t %d\t %d\t %d\n",
                       pID[i], aTime[i], bTime[i], cTime[i],
                       taTime[i], wTime[i], rTime[i]);
            }
        }
    }

    printf("Avg. TAT = %.2f\n", ((float)totalTAT / n));
    printf("Avg. WT = %.2f\n", ((float)totalWT / n));

    return 0;
}
```

Output:

```
Round Robin Scheduling


Enter number of processes: 5

Enter AT BT:
        P1: 0 5
        P2: 1 3
        P3: 2 1
        P4: 3 2
        P5: 4 3
Enter time quantum: 2

PID     AT      BT      CT      TAT     WT      RT
P1      0       5       13      13      8       0
P2      1       3       12      11      8       1
P3      2       1       5       3       2       2
P4      3       2       9       6       4       4
P5      4       3       14      10      7       5
Avg. TAT = 8.60
Avg. WT = 5.80
```

Program 2

Question:

WriteaC program to simulate a multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code:

```
#include <stdio.h>

intmain()
{
   printf("Multi-level Queue Scheduling\n");
   printf("RUSHIL
   SATHISH1WA24CS243\n");
   intn;
   printf("\nEnter number of processes: ");
   scanf("%d", &n);

   //arrival time = aTime        burst time = bTime
   //completion time = cTime     turnaround time = taTime
   //wait time = wTime
   //sys = system                usr = user

   intsysPID[n], sysAT[n], sysBT[n],sysCT[n],
      sysTAT[n], sysWT[n], sSize = 0;
   intusrPID[n], usrAT[n], usrBT[n],usrCT[n],
      usrTAT[n], usrWT[n], uSize = 0;
   intcurrTime = 0, totalTAT = 0, totalWT = 0, temp, type[n];

   for(int i = 0; i < n; i++)
   {
      printf("P%d: ", i + 1);

      printf("\n\tEnter process type (0 = System, 1 = User): ");
      scanf("%d", &type[i]);

      printf("\tEnter AT BT: ");
```

```
    if (type[i] == 0)
    {
        sysPID[sSize] = i + 1;
        scanf("%d%d",&sysAT[sSize], &sysBT[sSize]);
        sSize++;
    }
    else if (type[i] == 1)
    {
        usrPID[uSize] = i + 1;
        scanf("%d%d",&usrAT[uSize], &usrBT[uSize]);
        uSize++;
    }
}

// sorting sys processes by AT
for (int i = 0; i < sSize; i++)
{
    for(intj=0;j<sSize-i-1;j++)
    {
        if (sysAT[j] > sysAT[j + 1])
        {
            temp = sysPID[j];
            sysPID[j] = sysPID[j + 1];
            sysPID[j + 1] = temp;

            temp = sysAT[j];
            sysAT[j] = sysAT[j + 1];
            sysAT[j + 1] = temp;

            temp = sysBT[j];
            sysBT[j] = sysBT[j + 1];
            sysBT[j + 1] = temp;
        }
    }
}

// sorting user processes by AT
for (int i = 0; i < uSize; i++)
{
    for(intj=0;j<uSize-i-1;j++)
    {
        if (usrAT[j] > usrAT[j + 1])
```

```
        {
            temp = usrPID[j];
            usrPID[j] = usrPID[j + 1];
            usrPID[j + 1] = temp;

            temp = usrAT[j];
            usrAT[j] = usrAT[j + 1];
            usrAT[j + 1] = temp;

            temp = usrBT[j];
            usrBT[j] = usrBT[j + 1];
            usrBT[j + 1] = temp;
        }
    }
}

//systemqueue(higherpriority,executed first)
for(inti= 0; i < sSize; i++)
{
    if(currTime < sysAT[i])
        currTime = sysAT[i];

    sysCT[i] = currTime + sysBT[i];

    sysTAT[i] = sysCT[i] - sysAT[i];
    totalTAT += sysTAT[i];

    sysWT[i] = sysTAT[i] - sysBT[i];
    totalWT += sysWT[i];

    currTime = sysCT[i];
}

//userqueue(lowerpriority,executed next)
for(inti= 0; i < uSize; i++)
{
    if(currTime < usrAT[i])
        currTime = usrAT[i];

    usrCT[i] = currTime + usrBT[i];

    usrTAT[i] = usrCT[i] - usrAT[i];
```

```c
        totalTAT += usrTAT[i];

        usrWT[i] = usrTAT[i] - usrBT[i];
        totalWT += usrWT[i];

        currTime = usrCT[i];
    }

    printf("\nPID\t AT\t BT\t CT\t TAT\t WT\n");

    for (int j = 1; j < n + 1; j++)
    {
        for (int i = 0; i < sSize; i++)
        {
            if (sysPID[i] == j)
            {
                printf("P%d\t %d\t %d\t %d\t %d\t %d\n",
                       sysPID[i], sysAT[i], sysBT[i],
                       sysCT[i], sysTAT[i], sysWT[i]);
            }
        }

        for (int i = 0; i < uSize; i++)
        {
            if (usrPID[i] == j)
            {
                printf("P%d\t %d\t %d\t %d\t %d\t %d\n",
                       usrPID[i], usrAT[i], usrBT[i],
                       usrCT[i], usrTAT[i], usrWT[i]);
            }
        }
    }

    printf("Avg. TAT = %.2f\n", ((float)totalTAT / n));
    printf("Avg. WT = %.2f\n", ((float)totalWT / n));

    return 0;
}
```

Output:

```
Multi-level Queue Scheduling


Enter number of processes: 4
P1:
        Enter process type (0 = System, 1 = User): 0
        Enter AT BT: 0 4
P2:
        Enter process type (0 = System, 1 = User): 0
        Enter AT BT: 0 3
P3:
        Enter process type (0 = System, 1 = User): 1
        Enter AT BT: 0 8
P4:
        Enter process type (0 = System, 1 = User): 0
        Enter AT BT: 10 5

PID     AT      BT      CT      TAT     WT
P1      0       4       4       4       0
P2      0       3       7       7       4
P3      0       8       23      23      15
P4      10      5       15      5       0
Avg. TAT = 9.75
Avg. WT = 4.75
```

Program 3

Question:

WriteaCprogram to simulate Real-Time CPU Scheduling algorithms:

a) Rate-Monotonic

b) Earliest-deadline First

c) Proportional scheduling

a) Rate Monotonic Scheduling

Code:

```
#include <stdio.h>

typedef struct Task
{
   int   tID;
   int   cap;
   int    per;
   int   rem;
   int  next;
   int prio;
} Task;

// gcd(m, n) = gcd(n, m%n)
int gcd(int a, int b)
{
   while (b != 0)
   {
      int temp = b;
      b=a%b;
      a = temp;
   }

   return a;
}

int lcm(int a, int b)
{
   return((a*b)/gcd(a, b));
}
```

```
int main()
{
    printf("RateMonotonicScheduling\n");
    printf("Shorterperiod,higherpriority.\n");
    printf("RoydenMiranda1WA24CS243\n");

    int n;
    printf("\nEnternumberoftasks: ");
    scanf("%d", &n);

    Task tasks[n], temp;
    int hyper, currTime = 0;

    printf("EnterCapacityPeriod:\n");
    for (int i = 0; i < n; i++)
    {
        tasks[i].tID = i + 1;

        printf("\tT%d:",tasks[i].tID);
        scanf("%d%d",&tasks[i].cap, &tasks[i].per);

        tasks[i].rem = 0;
        tasks[i].next = 0;
    }

    // sorting tasks by period (priority)
    for (int i = 0; i < n; i++)
    {
        for(intj=0;j<n-i-1;j++)
        {
            if(tasks[j].per>tasks[j+1].per)     // if period of current task is higher
            {
                // swap task
                temp = tasks[j];
                tasks[j] = tasks[j + 1];
                tasks[j + 1] = temp;
            }
        }
    }

    // assigning priority
    for (int i = 0; i < n; i++)
```

```
    tasks[i].prio = tasks[i].per;

// defining hyperperiod
hyper = tasks[0].per;
for(inti=0; i <n;i++)
    hyper=lcm(hyper,tasks[i].per);

printf("Hyperperiod=%d\n", hyper);

printf("\nTime\tRunningTask\n");

for(currTime =0;currTime < hyper; currTime++)
{
    for(inti= 0;i<n;i++)
    {
        if(currTime%tasks[i].per == 0)
        {
            tasks[i].rem=tasks[i].cap;
            tasks[i].next+=tasks[i].per;
        }
    }

    intcurrTask = -1;

    for(inti= 0; i < n; i++)
    {
        if(tasks[i].rem > 0)
        {
            currTask = i;
            break;
        }
    }

    if(currTask != -1)
    {
        printf("%d\t T%d\n", currTime, tasks[currTask].tID);
        tasks[currTask].rem--;
    }
    else
        printf("%d\t Idle\n", currTime);
}
```

```
    return 0;
}
```

Output:

```
Rate Monotonic Scheduling
Shorter period, higher priority.


Enter number of tasks: 2
Enter Capacity Period:
        T1: 2 5
        T2: 3 10
Hyperperiod = 10

Time     Running Task
0        T1
1        T1
2        T2
3        T2
4        T2
5        T1
6        T1
7        Idle
8        Idle
9        Idle
```

b) Earliest Deadline First Scheduling

Code:

```
#include <stdio.h>

typedef struct Task
{
   int tID;
   int cap;
   int per;
   int dLine;
   int rem;
   int next;
   int absDLine;
} Task;

// gcd(m, n) = gcd(n, m%n)
int gcd(int a, int b)
{
   while (b != 0)
   {
      int temp = b;
      b=a%b;
      a = temp;
   }

   return a;
}

int lcm(int a, int b)
{
   return((a*b)/gcd(a, b));
}

int main()
{
   printf("EarlistDeadline First Scheduling\n");
   printf("RUSHIL
   SATHISH1WA24CS243\n");
   int n;
   printf("\nEnternumber of tasks: ");
```

```
scanf("%d", &n);

Task tasks[n];
int hyper = 1, currTime = 0;

printf("EnterCapacityPeriodDeadline: \n");
for (int i = 0; i < n; i++)
{
   tasks[i].tID = i + 1;

   printf("\tT%d:",tasks[i].tID);
   scanf("%d%d%d",&tasks[i].cap, &tasks[i].per, &tasks[i].dLine);

   tasks[i].rem = 0;
   tasks[i].next = 0;
   tasks[i].absDLine = 0;
}

// defining hyperperiod
for (int i = 0; i < n; i++)
   hyper=lcm(hyper,tasks[i].per);

printf("Hyperperiod=%d\n",hyper);

printf("\nTime\tRunningTask\n");

for(currTime=0;currTime<hyper; currTime++)
{
   for (int i = 0; i < n; i++)
   {
      if(currTime==tasks[i].next)
      {
         tasks[i].rem=tasks[i].cap;
         tasks[i].absDLine=currTime + tasks[i].dLine;
         tasks[i].next+=tasks[i].per;
      }
   }

   int currTask = -1;

   for (int i = 0; i < n; i++)
   {
```

```c
            if (tasks[i].rem > 0)
            {
                if (currTask == -1 || tasks[i].absDLine < tasks[currTask].absDLine)
                {
                    currTask = i;
                }
            }
        }

        if (currTask != -1)
        {
            printf("%d\t T%d\n", currTime, tasks[currTask].tID);
            tasks[currTask].rem--;
        }
        else
            printf("%d\t Idle\n", currTime);
    }

    return 0;
}
```

Output:

```
Earlist Deadline First Scheduling


Enter number of tasks: 3
Enter Capacity Period Deadline:
        T1: 1 2 2
        T2: 1 5 5
        T3: 2 10 10
Hyperperiod = 10

Time     Running Task
0        T1
1        T2
2        T1
3        T3
4        T1
5        T2
6        T1
7        T3
8        T1
9        Idle
```

c) Proportional Share Scheduling

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct Process
{
    int pID;
    int tix;
} Process;

int main()
{
    printf("Proportional Share Scheduling\n");
    printf("RUSHIL
    SATHISH1WA24CS243\n");
    int n;
    printf("\nEnter number of processes: ");
    scanf("%d", &n);

    Process prcss[n];
    srand(time(NULL));
    int totalTix = 0;

    printf("Enter tickets: \n");
    for (int i = 0; i < n; i++)
    {
        prcss[i].pID = i + 1;

        printf("\tP%d: ", prcss[i].pID);
        scanf("%d", &prcss[i].tix);

        totalTix += prcss[i].tix;
    }

    int timePer;
    printf("Enter time period: ");
    scanf("%d", &timePer);
    printf("\n");
```

```
    for(inti=0; i < timePer; i++)
    {
        intwinTicket = rand() % totalTix + 1;
        intticketPool = 0;
        intwinIndex = 0;

        for(intj= 0; j < n; j++)
        {
            ticketPool += prcss[j].tix;
            if(winTicket <= ticketPool)
            {
                winIndex = j;
                break;
            }
        }

        printf("Time: %d\n Ticket picked: %d Winner: P%d\n", i, winTicket, prcss[winIndex].pID);
    }

    printf("\n\tScheduling Summary \n");
    for(inti=0; i < n; i++)
    {
        floatshare = ((float)prcss[i].tix / totalTix) * 100;
        printf("P%d gets %.2f%% of processor time.\n", prcss[i].pID, share);
    }

    return 0;
}
```

Output:

```
Proportional Share Scheduling


Enter number of processes: 3
Enter tickets:
        P1: 10
        P2: 20
        P3: 30
Enter time period: 5

Time: 0
 Ticket picked: 34    Winner: P3
Time: 1
 Ticket picked: 2    Winner: P1
Time: 2
 Ticket picked: 20    Winner: P2
Time: 3
 Ticket picked: 42    Winner: P3
Time: 4
 Ticket picked: 3    Winner: P1

        Scheduling Summary
P1 gets 16.67% of processor time.
P2 gets 33.33% of processor time.
P3 gets 50.00% of processor time.
```

Program 4

Question:

WriteaCprogram to simulate:

a) Producer-Consumer problem using semaphores.

b) Dining-Philosopher's problem

a) Producer-Consumer Problem using Semaphores

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty;          // counts empty slots
sem_t full;           // counts filled slots
pthread_mutex_t mutex; // for mutual exclusion

void *producer(void *arg)
{
   int item, i = 0;

   while (1)
   {
      item = i++; //produceanitem(herejustincrementing number)

      sem_wait(&empty);          // wait for empty slot
      pthread_mutex_lock(&mutex); //entercriticalsection

      buffer[in] = item;
      printf("Produced: %d at buffer[%d]\n", item, in);

      in = (in + 1) % BUFFER_SIZE;
      pthread_mutex_unlock(&mutex); //leavecriticalsection
```

```
        sem_post(&full);            //signalthatnewitemisavailable
        sleep(1);                   // simulate time to produce
    }
}

void *consumer(void *arg)
{
    int item;

    while (1)
    {
        sem_wait(&full);            // wait for filled slot
        pthread_mutex_lock(&mutex); // enter critical section

        item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out);

        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex); //leavecriticalsection
        sem_post(&empty);           // signal that a slot is free
        sleep(2);                   // simulate time to consume
    }
}

int main()
{
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod,   NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);

    pthread_mutex_destroy(&mutex);
```

```
    return 0;
}
```

Output:

```
Producer-Consumer Problem with Semaphores


Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
```

b) Dining-Philosophers Problems

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5 // Number of philosophers

pthread_mutex_tforks[N]; // One mutex per fork
pthread_tphilosophers[N];

void*philosopher(void *num)
{
   intid=*(int*)num;
   intleft=id;            // left fork index
   intright=(id+1)% N; // right fork index

   while (1)
   {
      printf("Philosopher %d is thinking.\n", id);
      sleep(1); // Thinking

      //Toavoiddeadlock, philosophers with even id pick left then right,
      // with odd id pick right then left.
      if (id % 2 == 0)
      {
         // Pick up left fork
         pthread_mutex_lock(&forks[left]);
         printf("Philosopher%d picked up left fork %d.\n", id, left);

         // Pick up right fork
         pthread_mutex_lock(&forks[right]);
         printf("Philosopher%d picked up right fork %d.\n", id, right);
      }

      else
      {
         //Pickupright fork
         pthread_mutex_lock(&forks[right]);
         printf("Philosopher %d picked up right fork %d.\n", id, right);
```

```
            //Pickup left fork
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher%d picked up left fork %d.\n", id, left);
        }

        // Eating
        printf("Philosopher%diseating.\n", id);
        sleep(2); //Put down forks

        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);

        printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
    }
}

int main()
{
    int i;
    intids[N];//Initialize forks (mutexes)

    for(i=0;i<N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }

    //Createphilosopher threads
    for (i = 0; i < N; i++)
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);

    //Jointhreads (will never happen here, but good practice)
    for (i = 0; i < N; i++)
        pthread_join(philosophers[i], NULL);

    //Destroymutexes (unreachable here)
    for (i = 0; i < N; i++)
        pthread_mutex_destroy(&forks[i]);

    return 0;
}
```

Output:

```
Dining Philosophers Problem


Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 1 is thinking.
Philosopher 0 is thinking.
Philosopher 2 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 4 picked up left fork 4.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
```

Program 5

Question:

WriteaCprogram to simulate:

a) Bankers' algorithm for the purpose of deadlock avoidance.

b) Deadlock Detection

a) Safety Algorithm

Code:

```
#include <stdio.h>

intprs;//numberofprocesses
intrcs;//numberofresources

voidsafety(intalloc[prs][rcs], int max[prs][rcs], int avail[rcs])
{
    int need[prs][rcs];
    int work[rcs];
    int finish[prs];
    int seq[prs];
    int done = 0;

    //initneed=max - alloc
    for(inti=0;i<prs; i++)
        for(intj=0;j<rcs; j++)
            need[i][j]=max[i][j]-alloc[i][j];

    // init work = available
    for(inti=0;i<rcs; i++)        // [i0 i1 i2 ...]
        work[i]=avail[i];

    //initfinish[i]=falseforall i
    for(inti=0;i<prs; i++)
        finish[i] = 0;

    while (done < prs)
    {
        //findingsuitable i
        int found = 0;
```

```
        for (int i = 0; i < prs; i++)
        {
            if (!finish[i]) // check finish[i] = false
            {
                int check = 1;
                for (int j = 0; j < rcs; j++)
                {
                    if (need[i][j] > work[j]) //checkneed_i <= work
                    {
                        check = 0;
                        break;
                    }
                }

                if (check)
                {
                    for (int k = 0; k < rcs; k++)
                        work[k] += alloc[i][k];

                    finish[i] = 1;
                    seq[done++] = i;
                    found = 1;
                }
            }
        }
        if (!found)
            break;
    }
    if (done < prs)
        printf("\nSystem is in an unsafe state.\n");

    else
    {
        printf("\nSystem is in a safe state.\n");
        printf("Safe sequence: ");
        for (int i = 0; i < done; i++)
        printf("P%d ", seq[i]);
    }
}

int main()
{
```

```c
    printf("SafetyAlgorithm\n");
    printf("RUSHIL
    SATHISH1WA24CS243\n");
    printf("Enterthenumber of processes: ");
    scanf("%d", &prs);

    printf("Enterthenumber of resources: ");
    scanf("%d", &rcs);

    // m x n (prs x rcs) matrices
    int alloc[prs][rcs];
    int max[prs][rcs];
    int avail[rcs];

    printf("\nEnterAllocation: \n");
    for(inti=0;i<prs;i++)
        for(intj=0;j<rcs; j++)
            scanf("%d",&alloc[i][j]);

    printf("\nEnterMaxDemand: \n");
    for(inti=0;i<prs;i++)
        for(intj=0;j<rcs; j++)
            scanf("%d",&max[i][j]);

    printf("\nEnterAvailable Resources: \n");
    for(inti=0;i<rcs;i++)
        scanf("%d",&avail[i]);

    safety(alloc,max,avail);

    return 0;
}
```

Output:

```
Safety Algorithm


Enter the number of processes: 5
Enter the number of resources: 3

Enter Allocation:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Demand:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence: P1 P3 P4 P0 P2
```

Program 6

Question:

WriteaC program to simulate the following contiguous memory allocation techniques.

a)Worst-fit

b)Best-fit

c)First-fit

a) Worst Fit, Best Fit, and First Fit

Code:

```
#include<stdio.h>

voidfirstFit(int blockSize[], int m, int processSize[], int n)
{
   intallocation[n];      //storesprocess allocation
   intblockAllocated[m];//stores whether allocated or not

   // init allocation = -1
   for(inti=0; i < n;i++)
      allocation[i] = -1;
   //initallocated = 0
   for(intj=0; j < m;j++)
      blockAllocated[j] = 0;

   // choosing suitable block
   for(inti=0; i < n;i++)
   {
      for(intj= 0; j <m;j++)
      {
         if(blockSize[j]>=processSize[i] && blockAllocated[j] == 0)
         {
            allocation[i] = j;
            blockAllocated[j]=1;
            break;
         }
      }
   }

   printf("\nFirst Fit\n");
   printf("Process No.\tProcess Size\tBlock No.\n");
```

```
    for (int i = 0; i < n; i++)
    {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void bestFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];
    int blockAllocated[m];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int j = 0; j < m; j++)
        blockAllocated[j] = 0;

    for (int i = 0; i < n; i++)
    {
        int bestIdx = -1; // index of smallest fitting block
        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i] && blockAllocated[j] == 0)
            {
                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
                    bestIdx = j;
            }
        }

        if (bestIdx != -1)
        {
            allocation[i] = bestIdx;
            blockAllocated[bestIdx] = 1;
        }
    }

    printf("\nBest Fit\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++)
```

```
    {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];
    int blockAllocated[m];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int j = 0; j < m; j++)
        blockAllocated[j] = 0;

    for (int i = 0; i < n; i++)
    {
        int worstIdx = -1; // index of largest found block
        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i] && blockAllocated[j] == 0)
            {
                if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
                    worstIdx = j;
            }
        }

        if (worstIdx != -1)
        {
            allocation[i] = worstIdx;
            blockAllocated[worstIdx] = 1;
        }
    }

    printf("\nWorst Fit\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++)
    {
```

```
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

int main()
{
    printf("Contiguous Memory Allocation\n");
    printf("RUSHIL
    SATHISH1WA24CS243\n\n");
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int blockSize[m], blockCopy[m];
    printf("Enter sizes of %d memory blocks:\n", m);
    for (int i = 0; i < m; i++)
        scanf("%d", &blockSize[i]);

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int processSize[n];
    printf("Enter sizes of %d processes:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &processSize[i]);

    firstFit(blockSize,  m,  processSize,  n);
    bestFit(blockSize,  m,  processSize,  n);
    worstFit(blockSize, m, processSize, n);

    return 0;
}
```

Output:

```
Contiguous Memory Allocation


Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

First Fit
Process No.     Process Size    Block No.
1               212             2
2               417             5
3               112             3
4               426             Not Allocated

Best Fit
Process No.     Process Size    Block No.
1               212             4
2               417             2
3               112             3
4               426             5

Worst Fit
Process No.     Process Size    Block No.
1               212             5
2               417             2
3               112             4
4               426             Not Allocated
```

Program 7

Question:

Writea Cprogram to simulate page replacement algorithms.

a)FIFO

b)LRU

c)Optimal

a) FIFO, LRU, and Optimal Replacement

Code:

```
#include <stdio.h>
#include <stdbool.h>

#define MAX_PAGES 100

// display all frames
voidprintFrames(intframes[], int numFrames)
{
   printf("[");
   for(int i = 0; i < numFrames; i++)
      if(frames[i] != -1)
         printf("%d", frames[i]);
   printf("]\n");
}

// fifo page replacement
voidsimulateFIFO(intpages[], int numPages, int numFrames)
{
   printf("\nFIFO PageReplacement\n");

   intframes[numFrames]; // memory slots
   intpageFaults = 0;
   intnext = 0;

   // init frames to -1
   for(int i = 0; i <numFrames; i++)
      frames[i] = -1;

   for(int i = 0; i <numPages; i++)
```

```
    {
        int currPage = pages[i];
        printf("Page%d->",currPage);

        bool present = false;
        for(intj=0;j<numFrames; j++)
        {
            if(frames[j]==currPage)
            {
                present = true;
                break;
            }
        }

        if (!present)
        {
            frames[next]=currPage;
            next=(next+1)%numFrames;
            pageFaults++;
        }

        printFrames(frames,numFrames);
    }
    printf("TotalPageFaults(FIFO): %d\n", pageFaults);
}

// least recently used page replacement
voidsimulateLRU(intpages[], int numPages, int numFrames)
{
    printf("\nLRUPageReplacement\n");

    int frames[numFrames];
    int lastUsed[numFrames];
    int pageFaults = 0;

    for(inti=0;i<numFrames; i++)
    {
        frames[i] = -1;
        lastUsed[i] = -1;
    }

    for(inti=0;i<numPages; i++)
```

```
{
    int currPage = pages[i];
    printf("Page%d->",currPage);

    // choosing suitable index
    int currIdx = -1;
    for(intj=0;j<numFrames; j++)
    {
        if(frames[j]==currPage)
        {
            currIdx = j;
            break;
        }
    }

    if (currIdx != -1)
        lastUsed[currIdx] = i;
    else
    {
        pageFaults++;
        int emptySlot = -1;
        for(intj=0;j<numFrames; j++)
        {
            if (frames[j] == -1)
            {
                emptySlot = j;
                break;
            }
        }

        if (emptySlot != -1)
        {
            frames[emptySlot]=currPage;
            lastUsed[emptySlot]= i;
        }
        else
        {
            int lruIdx = 0;
            intoldestTime=lastUsed[0];
            for(intj=1;j<numFrames; j++)
            {
                if(lastUsed[j]<oldestTime)
```

```
                    {
                        oldestTime = lastUsed[j];
                        lruIdx= j;
                    }
                }
                frames[lruIdx] = currPage;
                lastUsed[lruIdx] = i;
            }
        }
        printFrames(frames, numFrames);
    }
    printf("TotalPageFaults (LRU): %d\n", pageFaults);
}

// optimal page replacement
voidsimulateOptimal(intpages[], int numPages, int numFrames)
{
    printf("\nOptimalPageReplacement\n");

    int frames[numFrames];
    int pageFaults = 0;

    for(inti=0;i<numFrames; i++)
        frames[i] = -1;

    for(inti=0;i<numPages; i++)
    {
        intcurrPage=pages[i];
        printf("Page%d-> ", currPage);

        boolpresent=false;
        for(intj=0;j<numFrames; j++)
        {
            if(frames[j]== currPage)
            {
                present=true;
                break;
            }
        }

        if (!present)
        {
```

```
pageFaults++;
int emptySlot = -1;
for(intj=0;j<numFrames; j++)
{
   if (frames[j] == -1)
   {
      emptySlot = j;
      break;
   }
}

if (emptySlot != -1)
   frames[emptySlot]=currPage;
else
{
   int replaceIdx = -1;
   int farthestUse = -1;

   for(intj=0;j<numFrames; j++)
   {
      int nextUse = -1;
      for(intk=i+1;k< numPages; k++)
      {
         if(pages[k]==frames[j])
         {
            nextUse = k;
            break;
         }
      }

      if (nextUse == -1)
      {
         replaceIdx = j;
         break;
      }

      if(nextUse>farthestUse)
      {
         farthestUse=nextUse;
         replaceIdx = j;
      }
   }
```

```
                frames[replaceIdx] = currPage;
            }
        }
        printFrames(frames, numFrames);
    }
    printf("Total Page Faults (Optimal): %d\n", pageFaults);
}

int main()
{
    printf("Page Replacement Algorithms\n");
    printf("RUSHIL
    SATHISH1WA24CS243\n\n");
    int numPages, numFrames;
    int pages[MAX_PAGES];

    printf("Enter number of pages: ");
    if (scanf("%d", &numPages) != 1 || numPages <= 0)
        return 1;

    printf("Enter page reference string:\n");
    for (int i = 0; i < numPages; i++)
        if (scanf("%d", &pages[i]) != 1)
            return 1;

    printf("Enter number of frames: ");
    if (scanf("%d", &numFrames) != 1 || numFrames <= 0)
        return 1;

    simulateFIFO(pages, numPages, numFrames);
    simulateLRU(pages, numPages, numFrames);
    simulateOptimal(pages, numPages, numFrames);

    return 0;
}
```

Output:

```
Page Replacement Algorithms
Royden Miranda 1WA24CS240

Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

FIFO Page Replacement
Page 7 -> [7]
Page 0 -> [70]
Page 1 -> [701]
Page 2 -> [201]
Page 0 -> [201]
Page 3 -> [231]
Page 0 -> [230]
Page 4 -> [430]
Page 2 -> [420]
Page 3 -> [423]
Page 0 -> [023]
Page 3 -> [023]
Total Page Faults (FIFO): 10

LRU Page Replacement
Page 7 -> [7]
Page 0 -> [70]
Page 1 -> [701]
Page 2 -> [201]
Page 0 -> [201]
Page 3 -> [203]
Page 0 -> [203]
Page 4 -> [403]
Page 2 -> [402]
Page 3 -> [432]
Page 0 -> [032]
Page 3 -> [032]
Total Page Faults (LRU): 9

Optimal Page Replacement
Page 7 -> [7]
Page 0 -> [70]
Page 1 -> [701]
Page 2 -> [201]
Page 0 -> [201]
Page 3 -> [203]
Page 0 -> [203]
Page 4 -> [243]
Page 2 -> [243]
Page 3 -> [243]
Page 0 -> [043]
Page 3 -> [043]
Total Page Faults (Optimal): 7
```